

Tìm hiểu ứng dụng của cây chữ cái trong thi tin học

Dong Huaxing
Trường THPT số 1 Thiệu Hưng, Chiết Giang

Tháng 12 năm 2008

Nguồn gốc: Tìm hiểu ứng dụng của cây chữ cái trong thi tin học

IOI China candidate team papers / 2009 / Dong Huaxing

Abstract

Cây chữ cái, còn gọi là cây tìm kiếm từ hoặc *trie*, có cấu trúc đơn giản và đặc biệt thích hợp cho các bài toán truy xuất liên quan tới chuỗi. Bài viết tổng kết kinh nghiệm học tập của tác giả, trình bày các ứng dụng thường gặp của *trie* trong tìm kiếm nhanh, sắp xếp chuỗi, giảm chuyển trạng thái vô ích, xử lý tiền tố chung dài nhất, đồng thời nêu một vài hướng mở rộng và giới hạn của cấu trúc này.

Từ khóa. Cây chữ cái; *trie*; truy xuất nhanh; sắp xếp chuỗi; chuyển trạng thái vô ích; tiền tố chung dài nhất.

Contents

1	Mở đầu	1
2	Kiến thức chuẩn bị	1
2.1	Phân tích một bài mẫu	1
2.2	Giới thiệu cấu trúc	2
3	Truy xuất nhanh trên chuỗi	3
4	Sắp xếp chuỗi	4
5	Giảm chuyển trạng thái vô ích	5
6	Tiền tố chung dài nhất	5
6.1	LCP của chuỗi	5
6.2	Tiền tố chung liên quan tới số	6
7	Mở rộng và giới hạn	7
7.1	Mở rộng ứng dụng	7
7.2	Giới hạn của <i>trie</i>	7
8	Tổng kết	7

1 Mở đầu

Khi vào phòng đọc của thư viện để tìm sách, ta thường dựa vào nhãn phân loại trên giá sách. Khi mở trình quản lý tệp của máy tính, ta cũng thấy các thư mục lồng nhau từng tầng. Những cấu trúc ấy giúp giải quyết một nhu cầu rất thường gặp trong đời sống: truy xuất và tìm kiếm.

Trong quá trình học thi tin học, ta cũng thường gặp những bài toán truy xuất. Cách liệt kê trực tiếp, không dùng cấu trúc dữ liệu, tuy dễ viết nhưng hay có hiệu quả thấp. Câu hỏi tự nhiên là: có cấu trúc nào đơn giản nhưng cải thiện được tốc độ truy xuất hay không?

Bài viết này giới thiệu cây chữ cái, một cấu trúc dữ liệu giúp xử lý tốt nhiều bài toán truy xuất liên quan tới chuỗi.

2 Kiến thức chuẩn bị

2.1 Phân tích một bài mẫu

Ví dụ 1: cây tìm kiếm từ. Cho một danh sách từ cố định. Cần xây dựng một cây tìm kiếm từ thỏa các điều kiện:

1. Nút gốc không chứa chữ cái; mỗi nút khác gốc chỉ chứa đúng một chữ cái tiếng Anh in hoa.
2. Với một nút bất kỳ, ghép các chữ cái trên đường đi từ gốc tới nút đó ta được từ tương ứng của nút. Mỗi từ trong danh sách phải là từ tương ứng của một nút nào đó.
3. Trong các cây thỏa hai điều kiện trên, số nút phải nhỏ nhất.

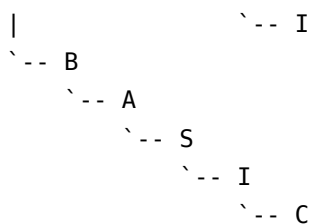
Cần đếm số nút của cây.

Với dữ liệu mẫu:

A
AN
ASP
AS
ASC
ASCII
BAS
BASIC

đáp án là 13. Cây có thể hình dung như sau:

```
Root
|-- A
|   |-- N
|       \-- S
|           |-- P
|               \-- C
|                   \-- I
```



Do yêu cầu số nút nhỏ nhất, tại mỗi nút mọi con mang cùng chữ cái phải được gộp lại. Vì thế ta chỉ cần mô phỏng quá trình chèn từng từ vào cây và đếm các nút được tạo mới.

2.2 Giới thiệu cấu trúc

Cấu trúc trong ví dụ trên chính là cây chữ cái, hay trie. Đây là một cây có gốc với các đặc điểm sau:

1. Gốc không chứa ký tự; mỗi nút khác gốc chứa một ký tự. Ký tự ở đây không nhất thiết là chữ cái, ta cũng có thể xem chữ số như một ký tự để xây trie.
2. Chuỗi tương ứng với một nút là chuỗi thu được khi nối các ký tự trên đường đi từ gốc tới nút đó. Mỗi chuỗi cần lưu được đánh dấu tại một nút.
3. Các con của cùng một nút có ký tự đôi một khác nhau.

Ví dụ danh sách

inform, informer, information, informatics, olympic, olympiad

có thể được lưu bằng trie. Hai chuỗi informer và information có sáu ký tự đầu giống nhau, nên phần tiền tố inform được lưu chung, chỉ phần còn lại mới tách nhánh. Nhờ tận dụng tiền tố chung, trie tiết kiệm số ký tự thực sự cần lưu.

Các thao tác chèn, xóa và tìm kiếm đều rất đơn giản. Với chuỗi độ dài m , ta đi từ gốc, ở bước thứ i chuyển tới con ứng với ký tự thứ i , rồi thực hiện thao tác cần thiết. Để cài đặt các cạnh con của mỗi nút, thường có ba cách:

1. Mỗi nút có một mảng kích thước bằng bảng chữ cái; chỉ số là ký tự, giá trị là mã số nút con.
2. Mỗi nút giữ một danh sách liên kết các con.
3. Dùng biểu diễn con trái, anh em phải.

Cách thứ nhất dễ viết và nhanh nhưng tốn bộ nhớ hơn. Cách thứ hai tiết kiệm bộ nhớ hơn nhưng tìm con chậm hơn. Cách thứ ba tiết kiệm nhất, nhưng viết và truy xuất phức tạp hơn. Khi làm bài, chỉ cần chọn biểu diễn phù hợp với giới hạn dữ liệu.

3 Truy xuất nhanh trên chuỗi

Trie sử dụng tốt tiền tố chung của các chuỗi. Vì nó lưu trữ tiết kiệm hơn trong nhiều trường hợp, ta cũng kỳ vọng việc truy xuất sẽ hiệu quả.

Ví dụ 2: tìm từ mới. Cho một bảng gồm N từ đã biết và một đoạn văn chỉ gồm chữ cái tiếng Anh thường. Hãy in mọi từ không thuộc bảng từ đã biết, theo thứ tự xuất hiện đầu tiên.

Ví dụ:

```
5
one
dollar
hundred
thousand
it
it costs one thousand one hundred and one yuan.
```

Kết quả:

```
costs
and
yuan
```

Cách trực tiếp là lưu bảng từ trong một mảng. Với mỗi từ của đoạn văn, duyệt toàn bộ mảng để kiểm tra xem nó đã xuất hiện hay chưa; nếu chưa thì in ra và thêm vào cuối mảng. Cách này rất dễ nghĩ, nhưng thời gian lớn.

Cách phổ biến hơn là dùng băm. Ta băm trước các từ đã biết, sau đó duyệt từng từ trong đoạn văn; nếu chưa có trong bảng băm thì in ra và chèn. Nếu hàm băm tốt và xử lý va chạm hợp lý, độ phức tạp gần với chi phí đọc dữ liệu.

Trie cho một lời giải khác sát với bản chất bài toán hơn. Chèn toàn bộ bảng từ đã biết vào trie và đánh dấu những nút tương ứng với một từ đã biết. Sau đó duyệt các từ trong đoạn văn, tìm nút tương ứng trong trie. Nếu nút chưa được đánh dấu, ta in từ đó, đồng thời chèn và đánh dấu nó. Khi dùng mảng con theo bảng chữ cái, mỗi ký tự chỉ được xử lý một số hằng lần, nên tổng thời gian đúng bằng bậc của kích thước dữ liệu vào, với hằng số nhỏ.

So với liệt kê trực tiếp, trie giảm mạnh số phép so sánh. So với băm, trie không cần thiết kế hàm băm và không có rủi ro do va chạm; các thao tác tìm và chèn đều đi theo từng ký tự một cách xác định.

4 Sắp xếp chuỗi

Từ điển tiếng Anh sắp các từ theo thứ tự từ điển. Vì trie tổ chức các nhánh theo ký tự, nó cũng có thể dùng để sắp xếp chuỗi.

Ví dụ 3: danh sách cảm ơn. Cho N tên tiếng Anh, mỗi tên là một từ. Hãy in các tên theo thứ tự từ điển tăng dần.

Ví dụ:

```
5
Reagan
Bush
Clinton
Bush
Obama
```

Kết quả:

```
Bush
Bush
```

Clinton
Obama
Reagan

Các thuật toán sắp xếp thông thường như chọn, nổi bọt, trộn hay nhanh đều dựa vào phép so sánh. Với số nguyên nhỏ, một phép so sánh tốn $\mathcal{O}(1)$, nên sắp xếp trộn có độ phức tạp $\mathcal{O}(N \log N)$. Nhưng với chuỗi, so sánh trực tiếp hai chuỗi có thể tốn $\mathcal{O}(|S|)$, làm tổng chi phí thành $\mathcal{O}(N|S| \log N)$ trong trường hợp xấu.

Với trie, ta chèn toàn bộ tên vào cây. Nếu mỗi nút lưu các con theo thứ tự ký tự tăng dần, thì một phép duyệt trước trên cây sẽ đi qua các chuỗi theo đúng thứ tự từ điển. Mỗi khi gặp nút được đánh dấu là kết thúc một tên, ta in chuỗi tương ứng. Nếu cùng một tên có thể xuất hiện nhiều lần, nút đó cần lưu thêm số lần xuất hiện để in đủ.

Sơ đồ của ví dụ trên có thể hiểu như sau:

```
Root
|-- B -- u -- s -- h          (2)
|-- C -- l -- i -- n -- t -- o -- n
|-- O -- b -- a -- m -- a
`-- R -- e -- a -- g -- a -- n
```

Con số (2) cho biết Bush xuất hiện hai lần.

Với cách này, mỗi chuỗi được đọc và in một lần. Duyệt cây tốn chi phí tỷ lệ với số nút nhân với kích thước bảng chữ cái nếu dùng mảng con cố định. Khi bảng chữ cái nhỏ, tổng độ phức tạp là tuyến tính theo tổng độ dài dữ liệu. Như vậy trie không chỉ hỗ trợ truy xuất nhanh mà còn tạo ra một kiểu sắp xếp đặc thù cho chuỗi.

5 Giảm chuyển trạng thái vô ích

Trie cung cấp khả năng truy xuất nhanh, và nếu tận dụng được điều này, nó cũng có ích trong truy hồi và quy hoạch động.

Ví dụ 4: giải mã văn bản. Cho một từ điển gồm N từ và một văn bản độ dài L không có dấu câu. Hỏi văn bản có bao nhiêu cách tách thành các từ trong từ điển. Kết quả lấy theo mô-đun 12345679.

Ví dụ:

```
5
ab
cb
bc
ba
a
abcba
```

Kết quả là 2.

Đặt $F[i]$ là số cách giải thích tiền tố kết thúc ở ký tự thứ i của văn bản, với $F[0] = 1$. Từ vị trí i , ta thử độ dài x của từ tiếp theo. Nếu đoạn từ $i + 1$ tới $i + x$ là một từ trong từ điển, thực hiện chuyển

$$F[i + x] \leftarrow F[i + x] + F[i].$$

Đáp án là $F[L]$. Vấn đề chính nằm ở việc kiểm tra một đoạn có phải là từ trong từ điển hay không.

Nếu so sánh từng từ của từ điển với từng đoạn văn bản, độ phức tạp có thể lên tới $\mathcal{O}(NL^2)$. Có thể dùng KMP để tìm trước mọi vị trí khớp của từng từ, hoặc dùng băm để so khớp chuỗi, nhưng tổng chi phí vẫn lớn và hằng số không nhỏ.

Với trie, ta chèn toàn bộ từ điển vào cây và đánh dấu các nút kết thúc từ. Sau đó với mỗi vị trí i , bắt đầu từ gốc trie và lần lượt đọc các ký tự $i + 1, i + 2, \dots$ của văn bản. Khi đi tới một nút được đánh dấu, ta thực hiện một chuyển DP tương ứng. Nếu tại ký tự hiện tại không còn cạnh con phù hợp, ta dừng ngay vòng thử độ dài cho vị trí i .

Độ phức tạp hình thức là $\mathcal{O}(LD)$, trong đó D là độ sâu trie, tức độ dài từ dài nhất. Điểm quan trọng là nhiều nhánh sẽ dừng rất sớm khi không có tiền tố tương ứng trong từ điển. Vì thế trie không chỉ thay thế phép kiểm tra chuỗi bằng truy xuất nhanh, mà còn cắt bỏ nhiều chuyển trạng thái không thể xảy ra.

6 Tiền tố chung dài nhất

Các ứng dụng ở trên nhấn mạnh khả năng truy xuất của trie. Bản chất của khả năng đó là cách trie xử lý tiền tố chung của các chuỗi. Vì vậy trie cũng liên quan trực tiếp tới bài toán tiền tố chung dài nhất, hay LCP.

6.1 LCP của chuỗi

Ví dụ 5: tiền tố chung dài nhất của hai chuỗi. Cho N chuỗi chữ cái thường và Q truy vấn. Mỗi truy vấn hỏi độ dài tiền tố chung dài nhất của hai chuỗi được chỉ định.

Ví dụ, với hai chuỗi:

abc

abd

thì LCP của chúng có độ dài 2. Nếu hỏi một chuỗi với chính nó, đáp án là độ dài chuỗi, chẳng hạn 3 với abc.

Một số cách giải có thể dùng các cấu trúc chuyên cho LCP hoặc cây hậu tố. Với trie, ta làm như sau: chèn tất cả chuỗi vào trie, rồi ghi lại nút kết thúc của mỗi chuỗi. Khi đó tiền tố chung dài nhất của hai chuỗi chính là đường đi chung từ gốc tới tổ tiên chung sâu nhất của hai nút kết thúc. Nói cách khác, độ dài LCP bằng độ sâu của LCA của hai nút đó.

Nhờ vậy bài toán LCP được chuyển thành bài toán LCA trên cây. Có nhiều cách xử lý LCA:

1. Dùng thuật toán Tarjan ngoại tuyến với cấu trúc tập rời nhau.
2. Lấy thứ tự Euler của trie, rồi chuyển LCA thành bài toán RMQ.
3. Tiền xử lý tổ tiên 2^k của mỗi nút, sau đó trả lời truy vấn bằng cách nhảy lên nhanh.

Trie đóng vai trò chiếc cầu nối: nó biến quan hệ tiền tố của chuỗi thành quan hệ tổ tiên trên cây.

6.2 Tiền tố chung liên quan tới số

Với chuỗi, tiền tố chung dẫn tới bài toán LCP như trên. Với số, ta cũng có thể xét một khái niệm tương tự khi nhìn phần thập phân như một chuỗi chữ số.

Ví dụ 6: bài toán xấp xỉ. Hai số thực được gọi là có cùng độ chính xác k nếu khi giữ lại đúng k chữ số sau dấu thập phân, hai số trở nên giống nhau. Giá trị k không vượt quá số chữ số thập phân ít hơn trong hai số. Giá trị k lớn nhất gọi là độ chính xác chung dài nhất. Cho N số thực không âm nhỏ hơn 1, hãy tìm một số x sao cho tổng độ chính xác chung dài nhất giữa x và mọi số đã cho là lớn nhất.

Ví dụ:

4
0.315
0.31
0.314
0.3

Một đáp án hợp lệ là:

0.315

Ở đây thao tác giữ chữ số là cắt trực tiếp, không làm tròn. Vì vậy ta lấy phần sau dấu thập phân của mỗi số như một chuỗi và xây trie từ các chuỗi đó. Đáp án tối ưu chắc chắn tương ứng với một nút nào đó của trie: nếu chọn một chuỗi không phải tiền tố xuất hiện trong cây, kéo nó về tiền tố gần nhất trên cây không làm giảm tổng đóng góp.

Khi xây trie, với mỗi nút lưu v , số chuỗi có tiền tố tương ứng với nút đó. Nếu chọn chuỗi của một nút làm phần thập phân của x , tổng độ chính xác chung dài nhất giữa x và các số đã cho bằng tổng các giá trị v trên đường đi từ gốc tới nút đó. Do đó chỉ cần duyệt toàn bộ trie, tích lũy tổng trên đường đi và cập nhật đáp án. Thời gian vẫn tuyến tính theo tổng số chữ số đọc vào.

Nếu bài toán yêu cầu làm tròn thay vì cắt trực tiếp, cách xử lý phức tạp hơn, nhưng vẫn có thể khai thác trie. Bài viết gốc đề trường hợp đó như một hướng suy nghĩ thêm.

7 Mở rộng và giới hạn

7.1 Mở rộng ứng dụng

Trong bài giải mã văn bản, ta đã thấy câu hỏi “một đoạn có phải là một từ hay không” thực chất là một bài toán khớp mẫu. Nhắc tới khớp mẫu, ta thường nghĩ tới KMP; còn trie lại mạnh ở việc truy xuất nhiều mẫu có chung tiền tố. Khi kết hợp hai tư tưởng này, ta thu được một cấu trúc mạnh hơn: máy tự động Aho–Corasick.

Bài viết này không trình bày chi tiết AC automaton. Điều cần nhấn mạnh là: trie không chỉ là một cấu trúc độc lập, mà còn là nền tảng cho nhiều cấu trúc xử lý chuỗi phức tạp hơn.

7.2 Giới hạn của trie

Trie có nhiều ưu điểm, nhưng không phải bài nào cũng nên dùng. Các bài thích hợp với trie thường có bảng chữ cái nhỏ, nên trong thực hành ta hay bỏ qua hằng số do kích thước bảng chữ cái gây ra. Nếu bảng chữ cái rất lớn, cách lưu mảng con cố định sẽ tốn bộ nhớ; nếu chuyển sang danh sách hoặc cấu trúc khác để tiết kiệm bộ nhớ, thời gian truy xuất mỗi ký tự lại tăng.

Vì vậy trước khi chọn trie, cần xét kỹ giới hạn dữ liệu, kích thước bảng chữ cái, yêu cầu thời gian và bộ nhớ. Đây cũng là bước cần thiết khi chọn bất kỳ cấu trúc dữ liệu nào.

8 Tổng kết

Trie là một cấu trúc dữ liệu dễ hiểu, dễ nghĩ, dễ viết và dễ dùng. Nó tận dụng tiền tố chung của các chuỗi để tiết kiệm lưu trữ, đồng thời có ưu thế lớn trong truy xuất.

Các ứng dụng của trie đều xoay quanh đặc điểm đó. Ngoài tìm kiếm cơ bản, trie có thể sắp xếp chuỗi, giảm các chuyển trạng thái vô ích trong truy hồi hoặc quy hoạch động, và chuyển bài toán tiền tố chung dài nhất thành bài toán tổ tiên chung gần nhất trên cây. Các cấu trúc dựa trên trie như AC automaton còn có phạm vi ứng dụng rộng hơn. Dĩ nhiên, trie không phải lời giải vạn năng; nó cần được chọn sau khi phân tích dữ liệu và ràng buộc của bài toán.

Tài liệu tham khảo

1. Liu Rujia, Huang Liang, *Nghệ thuật thuật toán và thi tin học*.
2. Yang Yi, *Một lớp ứng dụng của hash trong thi tin học*, tuyển tập đội tuyển quốc gia IOI 2007.
3. Thomas H. Cormen và cộng sự, *Introduction to Algorithms*.
4. Matrix67, *Giải thích chi tiết thuật toán KMP*, <http://www.matrix67.com/blog/archives/115>, 2006-11-29.
5. Guo Huayang, *Bài toán RMQ và LCA*, tuyển tập đội tuyển quốc gia IOI 2007.
6. Wang Yun, *Xây dựng, vận dụng và cải tiến đồ thị trie*, tuyển tập đội tuyển quốc gia IOI 2006.

A Bài gốc: cây tìm kiếm từ

Trong phân tích cú pháp, ta thường cần kiểm tra một từ có thuộc danh sách từ khóa hay không. Để tăng tốc tìm kiếm và định vị, có thể vẽ cây tìm kiếm từ tương ứng với danh sách từ. Cây có các đặc điểm:

1. Gốc không chứa chữ cái; mỗi nút khác gốc chứa đúng một chữ cái in hoa.
2. Ghép các chữ cái trên đường từ gốc tới một nút ta được từ tương ứng với nút đó; mỗi từ trong danh sách phải tương ứng với một nút.
3. Trong các cây thỏa hai điều kiện trên, số nút là nhỏ nhất.

Với danh sách:

A
AN
ASP
AS
ASC
ASCII
BAS
BASIC

cây tối thiểu có dạng đã mô tả ở ví dụ 1 và có 13 nút, kể cả gốc.

Dữ liệu vào. Tập vào là một danh sách từ, mỗi dòng chứa đúng một từ và ký tự xuống dòng. Mỗi từ chỉ gồm chữ cái tiếng Anh in hoa, độ dài không quá 63. Tổng kích thước tập không quá 32 KiB và có ít nhất một dòng dữ liệu.

Dữ liệu ra. In một số nguyên duy nhất: số nút của cây tìm kiếm từ tương ứng với danh sách.

Nguồn. Kỳ thi Olympic Tin học Thanh thiếu niên Toàn quốc lần thứ 17, vòng 2.

B Bài gốc: danh sách cảm ơn

Sau khi tốt nghiệp đại học, bạn X nhận được một công việc tốt ở công ty Y nhờ sự giúp đỡ của nhiều người. Gần Tết, X muốn mời họ ăn một bữa và lập một danh sách cảm ơn. Để tránh việc bạn bè hiểu nhầm rằng thứ tự tên thể hiện mức độ đóng góp, X muốn sắp các tên theo thứ tự từ điển tăng dần.

Dữ liệu vào. Dòng đầu là số N , số lượng tên. Tiếp theo là N dòng, mỗi dòng là một tên tiếng Anh, chữ cái đầu viết hoa, các chữ còn lại viết thường, không có khoảng trắng.

Dữ liệu ra. In N dòng, mỗi dòng một tên theo thứ tự từ điển tăng dần.

Ví dụ.

Input

5

Reagan

Bush

Clinton

Bush

Obama

Output

Bush

Bush

Clinton

Obama

Reagan

Nguồn. Không rõ.

Lời cảm ơn

Tác giả cảm ơn người thân luôn quan tâm, các thầy cô đã tận tình dìu dắt, các bạn bè đã động viên và giúp đỡ, cũng như Hội Tin học Trung Quốc và các huấn luyện viên đội tuyển quốc gia IOI 2009 đã tạo cơ hội để tác giả trình bày bài viết này.